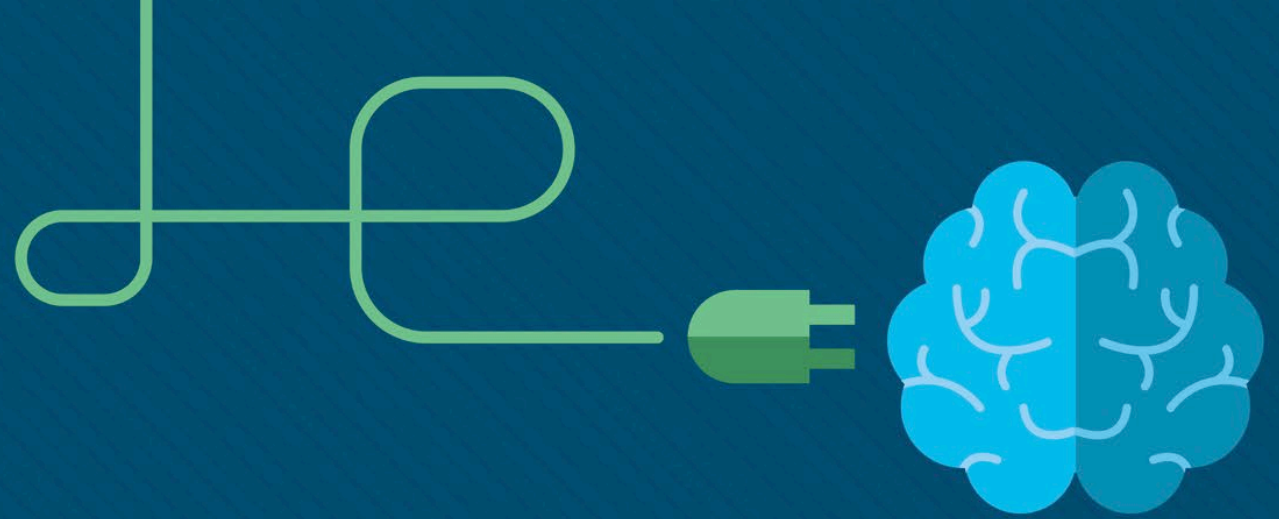




Week 5

RH124 Chapter 7. Controlling Access to Files

RH124 Chapter 8. Monitoring and Managing Linux Processes



1 Introduction

- In order to access or modify a file or directory, the correct permissions must be set. There are three different permissions that can be placed on a file or directory: read, write and execute.
- When listing a file with the `ls -l` command (for a directory, use an additional `-d` option), the output includes permission and ownership information:

```
[root@sads-lab ~]# ls -l /bin/ls
-rwxr-xr-x. 1 root root 140952 Jan 29 21:09 /bin/ls
[root@sads-lab ~]# ls -ld /usr/bin
dr-xr-xr-x. 2 root root 45056 Apr  9 09:15 /usr/bin
```

A refresher on the significance of the output of the `ls -l` command:

- File Type Field

```
-rwxr-xr-x. 1 root root 133792 Jan 18 2018 /bin/ls
```

- Permissions Field

```
-rwxr-xr-x. 1 root root 133792 Jan 18 2018 /bin/ls
```

- User Owner Field

```
-rwxr-xr-x. 1 root root 133792 Jan 18 2018 /bin/ls
```

- Group Owner Field

```
-rwxr-xr-x. 1 root root 133792 Jan 18 2018 /bin/ls
```

2 File Ownership

- By default, users own the file that they create. Although most commands usually associated the user owner as a name (eg, root in the previous example), the operating system associates the user ownership with the UID for that user (eg, the UID for root is zero, 0)
- Every file / directory also has a group owner. The default group owner of a file or directory is the primary group of the user who creates the file. Similarly, the group owner is usually associated with the group name but internally, the operating system associates the group ownership with the GID of the group.
- In the example below, you will notice that after the sysadmin creates the testfile, the user owner and group ownership of the file is automatically set to sysadmin user and sysadmin group (which is the primary group of the sysadmin user)

```
sysadmin@localhost:~$ touch testfile
sysadmin@localhost:~$ ls -l testfile
-rw-rw-r-- 1 sysadmin sysadmin 0 Nov 30 12:17 testfile
```

2 File Ownership (Cont'd)

- The `id` command is used to verify the UID of the user, primary group of the user and the GID of the primary group.

```
sysadmin@localhost:~$ id
uid=1001(sysadmin) gid=1001(sysadmin) groups=1001(sysadmin),4(adm),27(sudo)
```

- The above example shows that the UID of `sysadmin` is 1001, the primary group is `sysadmin` and the GID is 1001. The `id` command also list the group membership of the `sysadmin` account. `sysadmin` account belongs to the `adm` and `sudo` group in addition to the `sysadmin` group.

2.1 Changing File User Owner

- Using the chown command, the root user can change the owner of a file to an account other than the file creator.

```
chown [OPTION]... [OWNER] FILE...
```

- An example is shown below. Notice that the file owner has changed from sysadmin to root.

```
sysadmin@localhost:~$ cd Documents
sysadmin@localhost:~/Documents$ touch testfile
sysadmin@localhost:~/Documents$ ls -l testfile
-rw-rw-r-- 1 sysadmin sysadmin 0 Dec  1 04:56 testfile
sysadmin@localhost:~/Documents$ sudo chown root testfile
[sudo] password for sysadmin:
sysadmin@localhost:~/Documents$ ls -l testfile
-rw-rw-r-- 1 root sysadmin 0 Dec  1 04:56 testfile
```

2.1 Changing File Group Owner (Cont'd)

- Another option is for the user owner of the file to change the group owner by using the chgrp command. For example, you can use the chgrp command to change the file group owner of newfile2 from adm to sudo.

```
sysadmin@localhost:~/Documents$ sudo chgrp sudo newfile2
[sudo] password for sysadmin:
sysadmin@localhost:~/Documents$ ls -l newfile2
-rw-rw-r-- 1 sysadmin sudo 0 Dec  1 05:08 newfile2
sysadmin@localhost:~/Documents$
```

3 Understanding Permissions

- When viewing the long listing of a file, the permissions are broken into three sets of three characters:

User Owner permissions

Group Owner permissions

Other permissions

```
- rwx r-xr-x. 1 root root 133792 Jan 18 2018 /bin/ls
-rwx r-x r-x. 1 root root 133792 Jan 18 2018 /bin/ls
-rwxr-x r-x . 1 root root 133792 Jan 18 2018 /bin/ls
```

- Each permission group is granted three basic types of permissions: read, write and execute

User Owner			Group Owner			Other		
Read	Write	Execute	Read	Write	Execute	Read	Write	Execute
r	w	-	r	-	-	r	-	-

3 Understanding Permissions (Cont'd)

- The permissions have different meaning depending on whether they are applied to a file or directory.

Read

- The first character of each group represents the read permission. There is an r character if the group has the read permission, or a - character if the group does not.
 - On a file, this allows processes to read the contents of the file, meaning the contents can be viewed and copied.
 - On a directory, file names in the directory can be listed, but other details are not available.

Write

- The second character of each group represents the write permission. There is a w character if the group has the write permission, or a - character if the group does not.
 - A file can be written to by the process, so changes to a file can be saved. Note that w permission really requires r permission on the file to work correctly.
 - On a directory, files can be added to or removed from the directory. Note that w permission requires x permission on the directory to work correctly.

3 Understanding Permissions (Cont'd)

- On a directory, files can be added to or removed from the directory. Note that w permission requires x permission on the directory to work correctly.

Execute

- The third character of each group represents the execute permission. There is an x character if the group has the execute permission, or a - character if the group does not.
 - A file can be executed or run as a process.
 - On a directory, the user can use the cd command to "get into" the directory and use the directory in a pathname to access files and, potentially, subdirectories under this directory.

```
sysadmin@localhost:~$ ls -l /etc/passwd
-rw-r--r-- 1 root root 1602 May  2  2019 /etc/passwd
```

- For example, in the example above, the /etc/passwd file is owned by user root and group root. The root user have read and write access to the file but members in the root group have only read access to the contents of the file. All other users only have read access too. Do note that even though the root user is a member of the root group, the user owner permission overrides all other permission types.

3.1 Changing Basic File Permissions

- The `chmod` command is used to change the permissions of a file or directory. Only the root user or the user who owns the file is able to change the permissions of a file.

```
chmod MODE FILE...
```

- There are two techniques of changing permissions with the `chmod` command: symbolic and octal. The symbolic method is good for changing one set of permissions at a time. The octal method (also called the numeric method) requires knowledge of the octal value of each of the permissions and requires all three sets of permissions (user, group, other) to be specified every time. When changing more than one permission set, the octal method is probably the more efficient method.

3.1.1 Symbolic Method

- To use the symbolic method, use the symbols summarized in the tables below to specify the MODE argument, which represents which changes to make to the permissions.
- General format is `chmod <group><operation><permission>`

Symbol	Group
u	user owner
g	group owner
o	others
a	all (user owner, group owner, and others)

Symbol	Permission
r	read
w	write
x	execute

Symbol	Operation
+	add the permission, if necessary
=	specify the exact permission
-	remove the permission, if necessary

- Notice in the example below that the user owner of newfile has been granted execute permission on the file

```
sysadmin@localhost:~/Documents$ ls -l newfile
-rw-rw-r-- 1 sysadmin sysadmin 0 Dec  1 05:01 newfile
sysadmin@localhost:~/Documents$ chmod u+x newfile
sysadmin@localhost:~/Documents$ ls -l newfile
-rwxrw-r-- 1 sysadmin sysadmin 0 Dec  1 05:01 newfile
```

3.1.1 Symbolic Method (Cont'd)

- To remove the write permission from the group owner of newfile,

```
sysadmin@localhost:~/Documents$ chmod g-w newfile
sysadmin@localhost:~/Documents$ ls -l newfile
-rwxr--r-- 1 sysadmin sysadmin 0 Dec  1 05:01 newfile
```

- You can also execute more than one change to the permission of a file in a single command as follows:

```
sysadmin@localhost:~/Documents$ chmod u+x,g-w,o=rwx newfile
sysadmin@localhost:~/Documents$ ls -l newfile
-rwxr--rwx 1 sysadmin sysadmin 0 Dec  1 05:01 newfile
```

- In the example above, the user owner has been granted execute rights, the write access has been removed from the group owner, and all other accounts has been granted read, write and execute rights to newfile using a single chmod command.

3.1.1 Symbolic Method (Cont'd)

- You can also use the +X (capital X) and the -R option on a directory to “recursively” set the execute rights only for directory and its subdirectories (avoiding assigning execute rights on files).

```
[root@sads-lab ~]# ls -lR DirA
DirA:
total 12
drwxr--r--. 4 root root 4096 Apr 27 10:45 DirB1
drwxr--r--. 4 root root 4096 Apr 27 10:45 DirB2
-rw-r--r--. 1 root root  15 Apr 27 10:45 FileA

DirA/DirB1:
total 12
drwxr--r--. 2 root root 4096 Apr 27 10:45 DirC1
drwxr--r--. 2 root root 4096 Apr 27 10:45 DirC2
-rw-r--r--. 1 root root  15 Apr 27 10:45 FileB1

DirA/DirB1/DirC1:
total 4
-rw-r--r--. 1 root root 15 Apr 27 10:45 FileC1
```



```
[root@sads-lab ~]# chmod -R g+X DirA
[root@sads-lab ~]# ls -lR DirA
DirA:
total 12
drwxr-xr--. 4 root root 4096 Apr 27 10:45 DirB1
drwxr-xr--. 4 root root 4096 Apr 27 10:45 DirB2
-rw-r--r--. 1 root root  15 Apr 27 10:45 FileA

DirA/DirB1:
total 12
drwxr-xr--. 2 root root 4096 Apr 27 10:45 DirC1
drwxr-xr--. 2 root root 4096 Apr 27 10:45 DirC2
-rw-r--r--. 1 root root  15 Apr 27 10:45 FileB1

DirA/DirB1/DirC1:
total 4
-rw-r--r--. 1 root root 15 Apr 27 10:45 FileC1
```

Notice that only the subdirectories has the execute rights granted to the group owner. File permissions are not affected

3.1.2 Octal Method

- Using the octal method requires that the permissions for all three sets be specified. It is based on the octal numbering system in which each permission type is assigned a numeric value:

Octal Value	Permission
4	read
2	write
1	execute
0	none

- By adding together the combination of numbers from 0 to 7, any possible combination of *read*, *write*, and *execute* permissions can be specified for a single permission group set. For example in the table on the right.

Octal Value	Permission Set
7	<code>rwX</code>
6	<code>rw-</code>
5	<code>r-X</code>
4	<code>r--</code>
3	<code>-wX</code>
2	<code>-w-</code>
1	<code>--X</code>
0	<code>---</code>

3.1.2 Octal Method (Cont'd)

- In the below example, the octal method is used to assign the permission *rw-rw-r--* to the sample.txt file.

```
sysadmin@localhost:~$ touch sample.txt
sysadmin@localhost:~$ ls -l sample.txt
-rw-rw-r-- 1 sysadmin sysadmin 0 Dec  1 06:31 sample.txt
sysadmin@localhost:~$ chmod 764 sample.txt
sysadmin@localhost:~$ ls -l sample.txt
-rwxrw-r-- 1 sysadmin sysadmin 0 Dec  1 06:31 sample.txt
```

4 Permission Scenarios

- Recap the following:

Permission		Effect of File	Effect on Directory
read	<code>r</code>	Allows processes to read the contents of the file, meaning the contents can be viewed and copied.	File names in the directory can be listed, but other details are not available.
write	<code>w</code>	Allows for contents to be written to by the process, so changes to a file can be saved. Note that the <code>w</code> permission really requires the <code>r</code> permission on the file to work correctly.	Files can be added to or removed from the directory. Note that <code>w</code> permission requires <code>x</code> permission on the directory to work correctly.
execute	<code>x</code>	Allows for a file to be executed or run as a process.	Allows for the user to use the <code>cd</code> command to "get into" the directory and use the directory in a pathname to access files and, potentially, subdirectories under this directory.

4.1 Permission Scenarios (#1)

Question

Given the following information, what can the user `bob` do with the `/home/bob/test` file?

```
drwxr-xr-x 1 root root 4096 Apr 23 21:08 /  
drwxr-xr-x 1 root root 4096 Apr 17 22:31 /home  
drwxr-xr-x 1 bob  staff 4096 Apr 23 21:09 /home/bob  
-r--r--r-- 1 bob  staff 390  Apr 17 22:31 /home/bob/test
```

Answer

The user `bob` can view and copy the file because this user has the permissions of `r--` on the file.

4.2 Permission Scenarios (#2)

Question

Given the following information, can the user `bob` delete the `/home/bob/test` file?

```
drwxr-xr-x 1 root root 4096 Apr 23 21:08 /  
drwxr-xr-x 1 root root 4096 Apr 17 22:31 /home  
drwxr-xr-x 1 bob  staff 4096 Apr 23 21:09 /home/bob  
----- 1 bob  staff 390  Apr 17 22:31 /home/bob/test
```

Answer

Yes. While it may seem like the user `bob` shouldn't be able to delete this file because he has no permissions on the file itself, to delete a file a user needs to have *write* permission in the directory that the file is stored in. File permissions do not apply when deleting a file.

4.3 Permission Scenarios (#3)

Question

Given the following information, can the user `sue` view the contents of the `/home/bob/test` file?

```
drwxr-xr-x 1 root root 4096 Apr 23 21:08 /
drwxr-xr-x 1 root root 4096 Apr 17 22:31 /home
drwxr--r-- 1 bob  staff 4096 Apr 23 21:09 /home/bob
-r--r--r-- 1 bob  staff 390 Apr 17 22:31 /home/bob/test
```

Answer

No. Initially it may seem that the user `sue` should be able to view the contents of the `/home/bob/test` file because all users have *read* permission on the file.

However, when checking permissions, it is important to check the permissions on the parent directories of the file in question. In this case, only the user `bob` has the ability to "get into" the `/home/bob` directory because only the user `bob` has *execute* permission on that directory. If you can't get into a directory, it doesn't matter what the permissions are set to on the files in that directory.

4.4 Permission Scenarios (#4)

Question

Given the following information, who can delete the `/home/bob/test` file?

```
drwxr-xr-x 1 root root 4096 Apr 23 21:08 /  
drwxr-xr-x 1 root root 4096 Apr 17 22:31 /home  
drwxrwxrwx 1 bob staff 4096 Apr 23 21:09 /home/bob  
-rwx----- 1 bob staff 390 Apr 17 22:31 /home/bob/test
```

Answer

All users on the system can. Recall that to delete a file, a user needs *write* and *execute* permissions on the directory that the file is stored in. All users have *write* and *execute* permission on the `/home/bob` directory. This is a huge mistake because now all users can delete all files in the home directory of the user `bob`.

5 Changing Advanced File Permissions

- Special permissions, in addition to the basic Linux permissions of read, write and execute, are set by the administrator (root user) in situations where multiple users need to work routinely on the same directories and files. These permissions perform very specialized functions and are setuid, setgid and the sticky bit. They can be set using the chmod command, using either the symbolic or octal method.

Permission	Symbol	Symbolic	Octal Value	Purpose
setuid on a file	s	u+s	4000	Causes an executable file to execute under user owner identity, instead of the user running the command.
setgid on a file	s	g+s	2000	Causes an executable file to execute under group owner identity, instead of the user running the command.
setgid on a directory	s	g+s	2000	Causes new files and directories that are created inside to be owned by the group that owns the directory.
sticky on a directory	t	o+t	1000	Causes files inside a directory to be able to be removed only by the user owner, or the root user.

5.1 Setuid Permission

- When the *setuid* permission is set on an *executable binary file* (a program), the binary file is run as the owner of the file, not as the user who executed it. This permission is set on a handful of system utilities so that they can be run by normal users, but executed with the permissions of root, providing access to system files that the normal user doesn't normally have access to.
- An example of the use of the setuid permission is in the passwd command. The passwd command can be executed by normal users to update their passwords and hence needs to change the content of the /etc/shadow file. It makes use of the setuid permission to update the shadow file using root privileges because normal users do not have write access to the shadow file.

```
sysadmin@localhost:~$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1046 May  2  2019 /etc/shadow
sysadmin@localhost:~$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 59640 Jan 25  2018 /usr/bin/passwd
```

- Notice from the example above that the fourth character of the ls -l command output is an s instead of the usual x. It signifies that the setuid permission has been enabled. If it is an uppercase S, it means the execute permission has not been granted.

5.1 Setuid Permission (Cont'd)

- The setuid permission can be symbolically set by issuing the following command:

```
chmod u+s file
```

- To enable setuid permission using octal method, use the following:

```
chmod 4775 file
```

- To remove the setuid permission symbolically and using the octal method, use the following commands:

```
chmod u-s file
```

```
chmod 0775 file
```

5.2 Setgid Permission on a File

- When enabled on a file, the setgid permission is very similar to the setuid permission. It allows the file to be executed using the privilege of the group owner.

```
sysadmin@localhost:~$ ls -l /usr/bin/wall  
-rwxr-sr-x 1 root tty 30800 Oct 15 2018 /usr/bin/wall
```

- In the above example, when a user executes the wall command, the command is able to access files that are group owned by the tty group.
- This access is important because the wall command sends messages to terminals, which is accomplished by writing data to files like the following:

```
sysadmin@localhost:~$ ls -l /dev/tty?  
crw--w----. 1 root tty 4, 0 Mar 29 2013 /dev/tty0  
crw--w----. 1 root tty 4, 1 Oct 21 19:57 /dev/tty1
```

- Note that the tty group has write permission on the files above while users who are not in the tty group have no permissions on these files. Without the setgid permission, the wall command will fail.

5.3 Setgid Permission on a Directory

- When the setgid permission is set on a directory, files created in the directory are automatically owned by the group that owns the directory. In the sequence of commands, files and directories created within a setgid directory inherits the group owner and directories also inherits the setgid permission. Typically files created by a user will have its group owner be the primary group of the creator.

```
sysadmin@localhost:~$ ls -ld /tmp/data
drwxrwsrwx 2 root demo 4096 Dec  1 08:44 /tmp/data
sysadmin@localhost:~$ touch /tmp/data/tempfile
sysadmin@localhost:~$ ls -l /tmp/data/tempfile
-rw-rw-r-- 1 sysadmin demo 0 Dec  1 08:45 /tmp/data/tempfile
sysadmin@localhost:~$ mkdir /tmp/data/tempdir
sysadmin@localhost:~$ ls -ld /tmp/data/tempdir
drwxrwsr-x 2 sysadmin demo 4096 Dec  1 08:46 /tmp/data/tempdir
sysadmin@localhost:~$ touch /tmp/data/tempdir/tempfile2
sysadmin@localhost:~$ ls -l /tmp/data/tempdir/tempfile2
-rw-rw-r-- 1 sysadmin demo 0 Dec  1 08:46 /tmp/data/tempdir/tempfile2
```

5.4 Setting and Removing Setgid

- The setgid permission can be symbolically set by issuing the following command:

```
chmod g+s <file|directory>
```

- To enable setgid permission using octal method, use the following:

```
chmod 2775 <file|directory>
```

- To remove the setgid permission symbolically and using the octal method, use the following commands:

```
chmod g-s <file|directory>
```

```
chmod 0775 <file|directory>
```

5.5 Sticky Bit Permission

- The final advanced permission to discuss is the sticky permission, which is sometimes called the *sticky bit*. Setting this permission can be important to prevent users from deleting other user's files.
- Recall that to be able to delete a file, only the write and execute permissions on the directory are necessary. The write and execute permissions are also necessary to add files to a directory. So, if an administrator wants to make a directory where any user can add a file, the required permissions mean that the user can also delete any other user's file in that directory.
- The sticky permission on a directory modifies the write permission on a directory so that only specific users can delete a file within the directory:
 - The user who owns the file.
 - The user who owns the directory that has the sticky bit set (typically this is the root user).
 - The root user.
- On a typical Linux installation, there will normally be two directories that have the sticky permission set by default: the /tmp and /var/tmp directories. Besides user home directories, these two directories are the only locations that regular users have write permission for by default.

5.5 Sticky Bit Permission (Cont'd)

- As their name suggests, these directories are intended to be used for temporary files. Any user can copy files to the /tmp or /var/tmp directories to share with others.
- Since the /tmp or /var/tmp directories have the sticky permission, you don't have to worry about users deleting the shared files. However, there is a reason that these directories are considered temporary: these directories are automatically purged of their contents on a regular schedule. If you put a file in them, it will be removed automatically at some point in the future.
- To set the sticky bit permission symbolically, execute a command like the following:

```
chmod o+t <directory>
```

- To set the sticky bit permission numerically, add 1000 to the directory's existing permissions (assume the directory in the following example originally had 775 for its permissions):

```
chmod 1775 <file|directory>
```

5.5 Sticky Bit Permission (Cont'd)

- To remove the sticky permission symbolically, run:

```
chmod o-t <directory>
```

- To remove the sticky bit permission numerically, subtract 1000 from the directory's existing permissions:

```
chmod 0775 <directory>
```

6 Default File Permissions

- Linux sets the default permissions on new directories or files based upon the value of the creator's umask setting. The umask command is used to both set the umask value and display it.
- The umask command is automatically executed when a shell is started. To have a persistent setting for umask, a custom umask command can be added to the ~/.bashrc file.
- The umask value only affect the basic permissions and does not affect the special advanced permissions of setuid, setgid or sticky bit.
- When using the chmod command with octal values, we add together the values for the user, group and others. However, the umask octal value is used to specify permissions to be removed.
- The maximum default permissions for files and directories are as follows:

Files Type	Octal	Symbolic
Files	666	r w- r w- r w-
Directories	777	r wx r wx r wx

6 Default File Permissions (Cont'd)

- For example, with a umask value of 0022 for the sysadmin users, default permissions for new files and directories created by the sysadmin account is as shown below:

```
sysadmin@localhost:~$ umask
0002
sysadmin@localhost:~$ umask 022
sysadmin@localhost:~$ umask
0022
sysadmin@localhost:~$ touch testfile
sysadmin@localhost:~$ ls -l testfile
-rw-r--r-- 1 sysadmin sysadmin 0 Dec  1 10:01 testfile
sysadmin@localhost:~$ mkdir testdir
sysadmin@localhost:~$ ls -ld testdir/
drwxr-xr-x 2 sysadmin sysadmin 4096 Dec  1 10:01 testdir/
```

Note that since umask specifies the permissions to be removed, a umask digit of 2 converted to binary is 010. Hence, the write permission is to be removed from default permissions. With a umask value of 022, it means to have the write permission removed from both the group owner and others.

6 Default File Permissions (Cont'd)

- The following is the calculation of the default file permission given a umask value of 022:

Max File Default	666	rw-rw-rw-
Umask	-022	----W--W-
Result	644	rw-r--r--

- The following is the calculation of the default directory permission given the same umask value of 022:

Max Directory Default	777	rw-rw-rwx
Umask	-022	----W--W-
Result	755	rw-r-xr-x

6 Default File Permissions (Cont'd)

- The following shows another example when the umask is set to 077. Notice that all group owner and others permission are removed from the default file and directory permissions.

```
sysadmin@localhost:~$ umask 077
sysadmin@localhost:~$ umask
0077
sysadmin@localhost:~$ touch testfile
sysadmin@localhost:~$ ls -l testfile
-rw----- 1 sysadmin sysadmin 0 Dec  1 10:07 testfile
sysadmin@localhost:~$ mkdir testdir
sysadmin@localhost:~$ ls -ld testdir/
drwx----- 2 sysadmin sysadmin 4096 Dec  1 10:07 testdir/
```

6 Default File Permissions (Cont'd)

- The following is the calculation of the default file permission given a umask value of 077:

Max File Default	666	rw-rw-rw-
Umask	-077	---rwxrwx
Result	600	rw-----

- The following is the calculation of the default directory permission given the same umask value of 077:

Max Directory Default	777	rwxrwxrwx
Umask	-077	---rwxrwx
Result	700	rwx-----

7 Monitoring and Managing Linux Processes

- Students will learn about processes in the background or foreground, as well as how to make a process switch between the background and foreground. You will also be taught how to control processes by sending them signals using the *kill* command. In addition, different techniques for monitoring the resources that a process is using will be presented. You will see how to control the priority of processes to affect how much computing resources the processes will use. Finally, you will become familiar with utilities that allow multiple processes inside a single shell.

8 Process Control

- Executing a command results in something called a *process*. In the Linux operating system, processes are executed with the privileges of the user who executes the command. This allows for processes to be limited to certain capabilities based upon the user identity.
- In a nutshell, every process running on a linux computer runs under a system account.

```
sysadmin@localhost:~$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	4376	800	?	Ss	08:00	0:00	/sbin/init
root	7	0.0	0.0	78636	3688	?	S	08:00	0:00	/bin/login -f
syslog	10	0.0	0.0	191328	5752	?	Ss1	08:00	0:00	/usr/sbin/rsysl
root	13	0.0	0.0	28356	2752	?	Ss	08:00	0:00	/usr/sbin/cron
root	16	0.0	0.0	72296	3240	?	Ss	08:00	0:00	/usr/sbin/sshd
bind	26	0.8	0.0	1075996	39780	?	Ss1	08:00	0:01	/usr/sbin/named
sysadmin	62	0.0	0.0	19216	4164	?	S	08:01	0:00	-bash
sysadmin	82	0.0	0.0	34400	2916	?	R+	08:03	0:00	ps aux

9 Listing Processes

- The ps command can be used to list processes

```
ps [OPTION] ...
```

- The ps command supports three style of options:
 - Traditional UNIX style short options that use a single hyphen in front of a character
 - GNU style long options that use two hyphens in front of a word
 - BSD style options that use no hyphens and single character options
- The ps command will display the processes that can running in the current terminal by default and the output includes the following columns of information:

Column	Description
PID	The process identifier, which is unique to the process. This information is useful to control the process by its ID number.
TTY	The name of the terminal or pseudo-terminal where the process is running. This information is useful to distinguish between different processes that have the same name.
TIME	The total amount of processor time used by the process. Typically, this information isn't used by regular users.
CMD	The command that started the process.

9 Listing Processes (Cont'd)

- When the ps command is run with a BSD style option, the CMD column is replaced with the COMMAND column, which shows not just the command, but also its options and arguments. An additional column called STAT is also displayed, which conveys the state of the processes.

```
sysadmin@localhost:~$ ps
PID TTY          TIME CMD
 80 ?            00:00:00 bash
 94 ?            00:00:00 ps

sysadmin@localhost:~$ ps x
PID TTY          STAT TIME COMMAND
 80 ?            S    0:00 -bash
 95 ?            R+   0:00 ps x
```

Name	Flag	Description
Running	R	Task Running
	S	Task_Interruptible
	D	Task_Uninterruptible
	K	Task_Killable
Stopped	I	Task_Report_Idle
	T	Task_Stopped
	t	Task_Traced
Zombie	Z	Exit_Zombie
	X	Exit_Dead

There are several states that a process can be in ->

9 Listing Processes (Cont'd)

- Let's revisit the ps aux command.

```
sysadmin@localhost:~$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	4376	800	?	Ss	08:00	0:00	/sbin/init
root	7	0.0	0.0	78636	3688	?	S	08:00	0:00	/bin/login -f
syslog	10	0.0	0.0	191328	5752	?	Ss1	08:00	0:00	/usr/sbin/rsysl
root	13	0.0	0.0	28356	2752	?	Ss	08:00	0:00	/usr/sbin/cron
root	16	0.0	0.0	72296	3240	?	Ss	08:00	0:00	/usr/sbin/sshd
bind	26	0.8	0.0	1075996	39780	?	Ss1	08:00	0:01	/usr/sbin/named
sysadmin	62	0.0	0.0	19216	4164	?	S	08:01	0:00	-bash
sysadmin	82	0.0	0.0	34400	2916	?	R+	08:03	0:00	ps aux

- The command options of a, u and x means to allow ps command to show all processes, show processes by all users and ignore restrictions to only list the current user's processes and list all processes and remove the restriction to only display the processes that are running in the current terminal respectively.

9 Listing Processes (Cont'd)

- Instead of viewing just the processes running in the current terminal, you may want to view every process running on the system using the `-ef` option.
- The `ps` command is also commonly used together with the `grep` command to search for a specific process.

```
sysadmin@localhost:~$ ps aux | grep -i sshd
root          16  0.0  0.0  72292  3180 ?        Ss   23:03   0:00 /usr/sbin/sshd
sysadmin     92  0.0  0.0  11460  1004 ?        S+   23:10   0:00 grep --color=au
to -i sshd
```

- Alternatively, you can use the `pgrep` command to search for process by name and owner

```
sysadmin@localhost:~$ pgrep -li sshd
16 sshd
sysadmin@localhost:~$ pgrep -u sysadmin -l
70 bash
```


10 Background Processes

- So far we have been dealing with foreground processes. I.E, once executed, you need to wait for the command to finish its execution or issue a Ctrl+C to terminate the process.
- When a command may take some time to execute, it may be better to have that command execute in the *background*. When executed in the background, a child process releases control back to the parent process (the shell, in this case) immediately, allowing the user to execute other commands. To have a command execute as a background process, add the ampersand & character after the command.

```
COMMAND &
```

- For example, the command below executes the sleep command for 15 seconds in the background.

After your pressing of Enter, you notice that the sleep command runs in the background and the operating immediately returns the command prompt for your next command. Upon completion, the sleep command will display Done.

```
sysadmin@localhost:~$ sleep 15 &
[1] 303
sysadmin@localhost:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
sysadmin@localhost:~$
[1]+  Done                  sleep 15
```

10 Background Processes (Cont'd)

- In the previous example, you can see that once a background process is started, it outputs a job number followed by a space and then the process identification number (PID). These numbers can be used for controlling the process. The sleep command which was executed has a job number of 1 and a PID of 303.
- While there are still background processes being run in the terminal, they can be displayed by using the **jobs** command. It is important to point out that the **jobs** command will only show background processes in the current terminal. If background processes are running in another terminal, they will not be shown by running the **jobs** command in the current terminal. The following is an example of using the **jobs** command:

```
sysadmin@localhost:~$ sleep 2000 &
[1] 305
sysadmin@localhost:~$ sleep 3000 &
[2] 306
sysadmin@localhost:~$ jobs
[1]-  Running                  sleep 2000 &
[2]+  Running                  sleep 3000 &
```

- The plus sign to the right of the job number signifies the most recent background job while the minus sign means the previous job. All other jobs will have blanks. This allows you to see which is the most recent and next-most recent jobs.

10 Background Processes (Cont'd)

- To terminate the processes, send them to the foreground by using the fg command with the job number of the process to terminate and while the process is running in the foreground, use Ctrl+C to terminate the process.

```
sysadmin@localhost:~$ fg 1
sleep 2000
^C
sysadmin@localhost:~$ fg 2
sleep 3000
^C
sysadmin@localhost:~$ jobs
```

- You can also move a foreground process to the background by pressing the Ctrl+Z and the bg command as shown in the example on the right. For the bg command, please note that it is also possible to use the command as an argument.

- Format of the bg command:

```
bg [JOB_ID] ...
```

```
sysadmin@localhost:~$ sleep 1000
^Z
[1]+  Stopped                  sleep 1000
sysadmin@localhost:~$ bg 1
[1]+  sleep 1000 &
sysadmin@localhost:~$
```

11 Sending a Signal

- A signal is a message that is sent to a process to tell the process to take some sort of action, such as stop, restart, or pause. Signals are very useful in controlling the actions of a process.
- Some signals can be sent to processes by simple keyboard combinations. For example, to have a foreground process paused, send a *Terminal Stop* signal by pressing **Ctrl+Z**. A Terminal Stop pauses the program but does not completely stop the program. To completely stop a foreground process, send the *Interrupt* signal by pressing **Ctrl+C**.
- There are many different signals; each of them has a symbolic name and a numeric identifier. For example, **Ctrl+C** is assigned the symbolic name SIGINT and the numeric identifier of 2.
- To see a list of all the available signals for your system, execute kill -l command. A partial output is displayed below:

```
sysadmin@localhost:~$ kill -l
```

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP
6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1
11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM

11 Sending a Signal (Cont'd)

- Some of the more common signals are summarized in the following table:

Number	Full Name	Short Name	Purpose
1	SIGHUP	HUP	Hang up, usually ends a process
2	SIGINT	INT	Interrupt, usually ends a process
3	SIGQUIT	QUIT	Quit, usually ends a process
9	SIGKILL	KILL	Kill, forcefully ends a process
15	SIGTERM	TERM	Terminate, usually ends a process
18	SIGCONT	CONT	Continue, resumes a stopped process
19	SIGSTOP	STOP	Stop, forcefully stops a process
20	SIGTSTP	TSTP	Terminal stop, usually stops a process

11 Sending a Signal (Cont'd)

- There are several commands that will allow you to specify a signal to send to a process; the **kill** command is the most commonly used. The syntax for the **kill** command looks like the following:

```
kill [OPTIONS]... <pid>
```

- The **kill** command specifies which signal to be sent using options. It accepts three different ways to specify the signal:

```
-<signal>  
-s <signal>  
--signal <signal>
```

- The signal can be specified by using the number, the short name, or the full name. For example, to specify the *Interrupt* signal, any of the following options could be used:

```
-2  
-INT  
-SIGINT
```

- If the user doesn't specify a signal with an option, then the **kill** command defaults to sending the *Terminate* SIGTERM signal.

11 Sending a Signal (Cont'd)

- Example of sending a SIGTERM to a pid to kill the process is below:

```
sysadmin@localhost:~$ sleep 5000&
[1] 104
sysadmin@localhost:~$ pgrep sleep
104
sysadmin@localhost:~$ kill 104
sysadmin@localhost:~$ pgrep sleep
[1]+  Terminated                  sleep 5000
sysadmin@localhost:~$
```

- Sometimes a process may not end upon receiving a SIGTERM as it be behaving differently from the norm due to the way the programmer has created the code for the process. A user could try to use other signals, like SIGQUIT or SIGINT but these signals can also fail to end a process. If all else fails, the user can use a SIGKILL to force the process to end.
- It is recommended that the SIGKILL be used as a last resort because it forces the process to end immediately and will not allow the process the opportunity to “clean up” after itself. To kill a process using SIGKILL, issue the command `kill -9 <pid>` or `kill -KILL <pid>`

12 Monitoring the System

- The **uptime** command shows the current time and the amount of time the system has been running, followed by the number of users who are currently logged in and the load averages during the past one, five, and fifteen-minute intervals.
- To get a good idea of how busy a system is, use the **uptime** command:

```
sysadmin@localhost:~$ uptime  
07:08:23 up 6 days, 14:28, 1 user, load average: 0.13, 0.19, 0.18
```

- The numbers reported for the load averages are based upon how many CPU cores are available. Think of each processor as having 100% resources (CPU time) available. One processor = 100%, four processors = 400%. The **uptime** command is reporting the amount of resources used, but it is dividing by 100. So, 1.00 is actually 100% of the CPU time being used, 2.00 is 200% and so on.
- If a system has only one CPU core, then a value of 1.00 indicates that the system was fully loaded with tasks. If the system has two CPU cores, then a value of 1.00 would indicate a 50% load (1.00/2.00). An **uptime** reading of 1.00 (or 100%) usage on a 4 core CPU would imply that 1.00/4.00 (1/4 or 25%) of the CPU's total computational resources are being used.

12 Monitoring the System (Cont'd)

- To get an idea of how your system is using memory, the **free** command is helpful. This command displays not only the values for the random-access memory (RAM) but also for swap, which is space on the hard disk that is used as memory for idle processes when the random-access memory starts to become scarce.
- The **free** command reports the total amount of memory, as well as how much is currently being used and how much is free (available) for use. The output also breaks down the use of memory for buffers and caches:

```
sysadmin@localhost:~$ free
```

	total	used	free	shared	buff/cache	available
Mem:	65968580	3351268	57005400	3668	5611912	62240188
Swap:	2097148	659224	1437924			

13 Watching Processes

- There are many times when it's advantageous or informative to see a process execute over and over again. In those cases, the **watch** command can be used to monitor recurring processes by using the following syntax:

```
watch [OPTION]... COMMAND
```

- A command that would normally execute and exit could be used with the **watch** command to execute repeatedly. A good visual example would be to run the **watch** command with the **date** command to view the **date** command running repeatedly:

```
sysadmin@localhost:~$ watch date
```

```
Every 2.0s: date                                localhost: Mon Nov 25 23:17:57 2019
```

```
Mon Nov 25 23:17:57 UTC 2019
```

- By default, the **watch** command runs the given command every two seconds so the output of the **date** command will change to show a two-second difference.

13 Watching Processes (Cont'd)

- It is possible to change the interval desired for the watch command by using the -n option:

```
watch -n # [COMMAND...]
```

- Exit the watch command by pressing Ctrl+C to interrupt the command.

- To monitor changes to the last twenty lines of the log file /var/log/auth.log and execute the **tail** command every twenty seconds, use the following command:

```
sysadmin@localhost:~$ watch -n 5 tail -n 20 /var/log/auth.log
```

```
Every 5.0s: tail -n 20 /var/log/auth.log      localhost: Mon Nov 25 23:30:42 2019

Nov 25 23:03:59 localhost sshd[16]: Server listening on 0.0.0.0 port 22.
Nov 25 23:03:59 localhost sshd[16]: Server listening on :: port 22.
Nov 25 23:04:16 localhost login[7]: pam_unix(login:session): session opened for
user sysadmin by (uid=0)
Nov 25 23:04:16 localhost login[7]: pam_systemd(login:session): Failed to connec
t to system bus: No such file or directory
Nov 25 23:17:01 localhost CRON[97]: pam_unix(cron:session): session opened for u
ser root by (uid=0)
Nov 25 23:17:01 localhost CRON[97]: pam_unix(cron:session): session closed for u
ser root
```

13 Watching Processes (Cont'd)

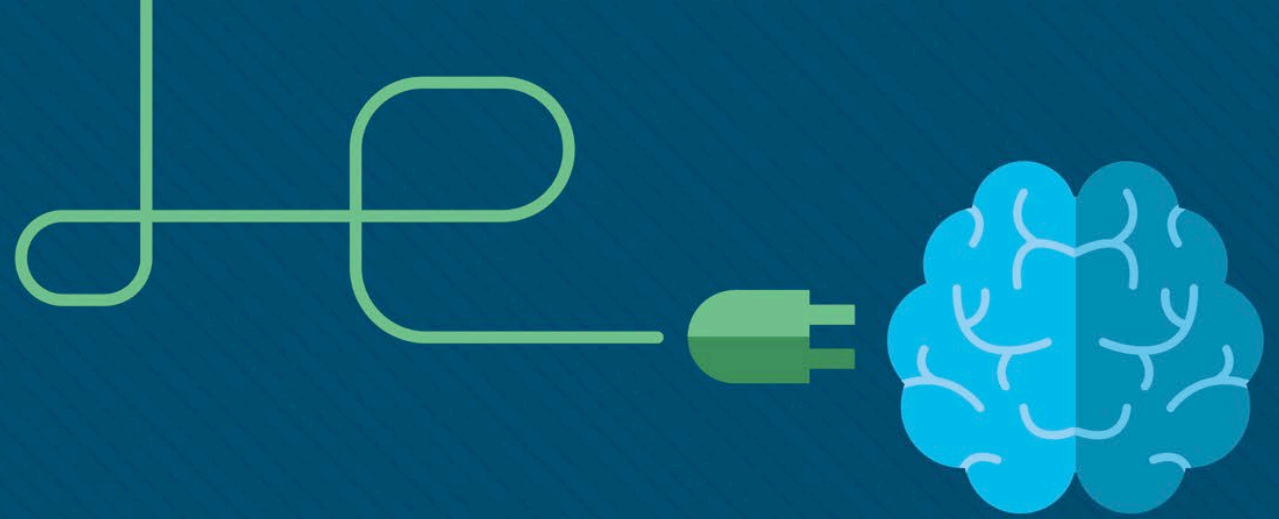
- You can also highlight any differences between each successive execution of the program that is being watched by specifying the **-d** option. In the example below, the **watch** command is used with the **-d** option to run the **free** command, used to display memory on the system, repeatedly:

```
sysadmin@localhost:~$ watch -d free
```

```
Every 2.0s: free                                  localhost: Mon Nov 25 23:35:04 2019
```

	total	used	free	shared	buff/cache	available
Mem:	65940536	17255436	43475144	4636	5209956	48180896
Swap:	999420	0	999420			

- The output above will show the amount of used, free, and available memory in the **Mem:** row updating every two seconds, and the differences in the memory will be highlighted in each new refresh of the screen.



Lab

RH124 Chapter 7.7 Lab Control Access to Files

RH124 Chapter 8.9 Lab Monitoring and Managing Linux Processes



Practice Exercises

1 File Ownership & Permissions

- Use `ls -l` to check the owner and group of a file. What do the columns represent?
- Verify your user ID, primary group, and additional groups using `id`.
- Change the owner of `testfile.txt` to another user.
- Change the group owner of `testfile.txt` to `developers`.

2 Modifying File Permissions

- Grant execute permission to the owner of `script.sh` using the symbolic method.
- Remove write permission from the group for `data.txt`.
- Set full permissions (`rwX`) for the owner and group, and read-only (`r--`) for others using octal notation.

3 Monitoring & Managing Processes

- Use `ps aux` to list running processes. What do the columns show?
- Find the process ID of a running `sleep` command using `pgrep`.
- Send a `SIGTERM` signal to stop the process.
- Start a long-running task in the background and bring it to the foreground.

4 System Monitoring

- Use `uptime` to check system load averages.
- Run `free -h` to check memory usage. What do the columns represent?
- Use `watch -n 5 free -h` to monitor memory usage every 5 seconds. How do you stop it?